



Tecnun
Universidad
de Navarra

ESCUELA DE INGENIERÍA
INGENIARITZA ESKOLA
SCHOOL OF ENGINEERING

C++ Laboratory Lesson

C++
For Engineering



Object-Oriented Programming (OOP)

Dr. Paul Bustamante / Dr. Jesús Paredes

Contenido

1.	Introduction	2
1.1	Exercise 1: My first namespace	2
1.2	Exercise 2: Namespace with classes	3
1.3	Exercise 3: Working with “strings”	3
1.4	Exercise 4: Sorting strings	4
1.5	Exercise 5: Vectors with standard data types (int, double)	4
1.6	Exercise 6: Vectors with objects	5
1.7	Exercise 7: Complete exercise	5
1.8	Exercise 8: Exceptions	6
	Additional exercises	7
1.9	Exercise 9: vector with strings	7
1.10	Exercise 10: exceptions	7
1.11	Exercise 11: vector and matrix	8
1.12	Exercise 12: vector and matrix with classes	9

1. Introduction

In this lesson, you'll use STL libraries, such as the **vector** and **string** classes. You'll also practice with C++ **namespaces** and **exceptions**. In some exercises, the code will be given, but it does not mean you only have to write it on your computer; you should try to complete it on your own.

1.1 Exercise 1: My first namespace

In this exercise, you will create your first namespace “**first_space**”, which will show you how you can work with namespaces. Create a new project with **VStudio**, copy the following code in the source file and run the program.

```
#include <iostream>
using namespace std;

// first name space
namespace first_space
{
    void func()
    {
        cout << "Inside first_space" << endl;
    }
}

using namespace first_space;
int main()
{
    func();    // calls directly the function
}
```

Lesson Nº 9 - C++ Programming Exercises

```
// Other way to call function from first space
first_space::func();

return 0;
}
```

Now, you must create a “**second_space**” with the same function **func()**. You must use it in the **main** program. Compile the project and run it, observing the result.

1.2 Exercise 2: Namespace with classes.

In this exercise you must create the same class “**product**” in two namespaces: one will be called **foodSpace** (for products like fruits, milk, etc.) and the other one must be **elecSpace** (for electronic devices, like TVs, Laptops, etc.).

The structure for the first class (inside **foodSpace** namespace) will be the following:

```
class product {
    double cost;
    char name[20];
public:
    product(const char* n, double p) : cost(p) { strcpy_s(name, n); }
    void prt() {
        cout << setw(20) << name << setw(5) << cost << endl;
    }
}
```

The class product in the **elecSpace** must have the number of **years** of warranty of the product. To test both namespaces, you must create a project with the following code in the main program:

```
//must include here all the necessary files
int main()
{
    foodSpace::product p1("Milk", 1.5);
    elecSpace::product tv1("LG 50", 750.99, 3); //TV with 3 years
    p1.prt();
    tv1.prt();

    return 0;
}
```

1.3 Exercise 3: Working with “strings”

In this exercise you must practice with the new class “**string**”. Create a new project and copy the following code and run it!

```
#include <iomanip>
#include <iostream>
#include <string> //Important
using namespace std;
int main()
{
    string frase1; //Expression
    cout << "Write a Sentence:";
    getline(cin, frase1);
    cout << "Backwards:" << endl;
    int len = frase1.size();
    for (int i = len - 1; i >= 0; i--) cout << frase1[i];

    string n1, n2;
    cout << "\nGive 2 names:"; cin >> n1 >> n2;
    if (n1 == n2) cout << "Equal names" << endl;
    else if (n1 < n2)
        cout << n1 << " is less than " << n2 << endl;
}
```

```
        else cout << n1 << " is greater than " << n2 << endl;
    string nt = n1 + n2;
    cout << "Together: " << nt << " Size=" << nt.size() << endl;
    cout << "\nEnd.." << endl;

    return 0;
}
```

1.4 Exercise 4: Sorting strings

In this exercise you must create an array of strings, like: `string vec[10];` .

Your program must ask for some words and store it in the array, then must print those words sorted alphabetically, ascendent or descendent, using the “>” or “<” operator (like do it with numbers).

1.5 Exercise 5: Vectors with standard data types (int, double)

In this exercise you must practice with the new class “**vector**”, using standard data types. Create a new project and copy the following code and run it!

```
#include <iostream>
#include <vector>
using namespace std;
int main()
{
    srand(time(NULL));

    vector<double> v1;
    int num = 0;
    cout << "Give num:";
    cin >> num;
    for (int i = 0; i < num; i++) {
        double val = rand() * 100.0 / RAND_MAX;
        v1.push_back(val);
    }
    int len = v1.size();
    for (int i = 0; i < len; i++) {
        cout << i << ":" << v1[i] << endl;
    }
    //otra forma
    int pos = 0;
    cout << "Erase?"; cin >> pos;
    v1.erase(v1.begin() + pos);
    cout << "-----\n";
    pos = 0;
    for (auto it = v1.begin(); it != v1.end(); it++, pos++) {
        cout << pos << ":" << *it << endl;
    }
    v1.clear();
    return 0;
}
```

1.6 Exercise 6: Vectors with objects

In this exercise you must learn how to create objects and store them in a **vector** container. Also, you will learn how to erase objects from different positions inside the container.

```
#include <iostream>
#include <vector>
using namespace std;

class vehicle
{
    string brand;
    int hp;
public:
    vehicle(string bd = "BYD", int h=100) : brand(bd),hp(h) { ; }
    void prt() { cout << brand << ", " << hp << endl; }
};

int main()
{
    vector<vehicle> lst;
    vehicle v1("Toyota", 150);
    lst.push_back(v1);
    lst.push_back(vehicle("vw", 115));
    lst.push_back(vehicle("mazda", 200));
    lst.push_back(vehicle()); //default
    lst.push_back(vehicle("fiat", 150));

    int len = lst.size();
    vector<vehicle>::iterator it1 = lst.begin();
    it1++;

    for (auto it = lst.begin(); it != lst.end(); it++) {
        vehicle v = *it;
        v.prt();
    }
    int pos = 0;
    cout << "Which object to erase?"; cin >> pos;
    cout << "Object to erase:";
    lst.at(pos).prt();

    lst.erase(lst.begin() + pos);

    len = lst.size();
    cout << "\n----- Other way..\n";
    for (int n = 0; n < len; n++) {
        vehicle v = lst[n];
        v.prt();
    }
    return 0;
}
```

1.7 Exercise 7: Complete exercise

Based on exercise 6, in this exercise you must create a new class **date** and use it in the class **vehicle** to put the date of the registration car.

```
class date
{
    int d, m, y;
public:
    date(); //constructor
    //more functions
}
```

```
class vehicle
{
    string brand;
    date reg; //car registration
    int hp;
public:
}
```

Lesson Nº 9 - C++ Programming Exercises

};

```
vehicle();  
void prt();  
//more functions  
};
```

You must create a program with a menu with 4 options:

- 1. Add Car (with all the data)
- 2. Print the list of Cars introduced in the data base.
- 3. Erase one car, asking for the position.
- 4. Find the cars registered between two dates (use months of 30 days)

You must use the **vector** container to store the cars.

1.8 Exercise 8: Exceptions

The following code shows how to catch an exception, in this case, for a negative number in the square root.

```
#include <iostream>  
using namespace std;  
double root(double n)  
{  
    if (n < 0)  
        throw exception(); //1. end function with exception  
    return sqrt(n);  
}  
int main()  
{  
    double val;  
    cout << "Give a number:";  
    cin >> val;  
    try  
    {  
        double r = root(val);  
        cout << "root=" << r << endl;  
    }  
    catch (...)  
    {  
        cerr << "Negative number.." << endl;  
    }  
    //If you do not know the throw type used in the try block,  
    // you can use the "three dots" syntax(...) inside the catch block,  
    // which will handle any type of exception:  
    cout << "End!\n";  
}
```

Additional exercises

1.9 Exercise 9: vector with strings

Create a function called **“words_to_phrase”** that takes an array of strings and the length of the array as input and returns a single **string** with all the words from the array concatenated together.

Create another function called **“words_to_phrase”** that takes a **vector** of **string** as input and returns a single **string** with all the words from the vector concatenated together.

Create a function called **“phrase_to_words”** that returns a **vector** of **string**, where each element contains a word from the phrase stored in the input **string** argument.

Create another function called **“phrase_to_words”** that returns an array of **string**, where each element contains a word from the phrase stored in the input **string** argument. This function should also return the length of the array via pass-by-reference.

```
void main()
{
    string words1[5]= { "red", "blue", "green", "yellow", "black" };
    vector<string> words2 = { "red", "blue", "green", "yellow", "black" };
    string phrase1 = words_to_phrase (words1,sizeof(words1)/sizeof(string));
    string phrase2 = words_to_phrase (words2);
    cout << phrase1 << endl << phrase2 << endl;

    int len;
    vector<string> words3 = phrase_to_words(phrase1);
    string *words4 = phrase_to_words(phrase2,len);
}
```

1.10 Exercise 10: exceptions

Consider the following class rectangle and main:

```
class rectangle
{
    double b;
    double h;
public:
    rectangle(double B = 2, double H = 1) { b = B; h = H; }
    void setBase(double _b)
    {
        try
        {
            if (_b <= 0)
                throw "b must be positive";
            else if (h > _b)
                throw "b must be greater than h";
            else
                b = _b;
        }
        catch (const char* msg){
            cerr << "ERROR: " << msg << endl;
        }
    }
    void prt() { cout << b << "\t" << h << endl; }
};
```

Lesson Nº 9 - C++ Programming Exercises

```
void main()
{
    rectangle r1;
    r1.prt(); cout << endl;

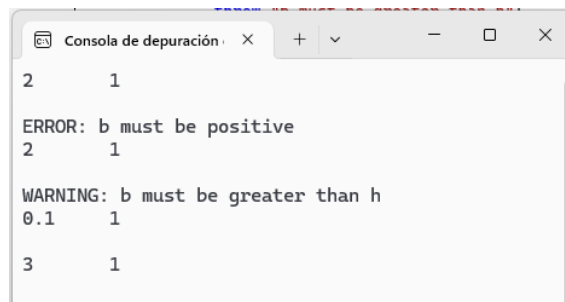
    r1.setBase(-5);
    r1.prt(); cout << endl;

    r1.setBase(0.1);
    r1.prt(); cout << endl;

    r1.setBase(3);
    r1.prt(); cout << endl;
}
```

Observe that, depending on the user's input, different error messages are displayed. This can be achieved using a try-catch sequence within another try-catch sequence. Modify the code in such a way that: consider "b" being null or negative as an external exception (from the external try-catch); consider "h > b" as an internal exception (from the internal try-catch).

The exercise has been designed in a way that when an exception occurs, the function terminates. Modify the code so that, if "h > b" occurs, the assignment is still made but a warning message is displayed. Below is the expected console output.



```
Consola de depuración: x + - - □ x
2      1
ERROR: b must be positive
2      1
WARNING: b must be greater than h
0.1    1
3      1
```

If it's easier for you, you can use "`std::error_code`" in the "`throw`" statements. For example:

```
try{
    if (_b <= 0)
        throw 1;
}
catch (int error_code){
    if (error_code == 1)
        cerr << "ERROR: b must be positive" << endl;
}
```

1.11 Exercise 11: vector and matrix

Imagine the rows and columns of a boolean matrix as corresponding to the seating arrangement of a bus. Consider the following example where a boolean matrix is created using one `vector` inside another:

```
void main()
{
    vector<vector<bool>> m(10, vector<bool>(4));
    for (int i = 0; i < m.size(); i++)
    {
        for (int j = 0; j < m[0].size(); j++)
        {
            m[i][j] = 0;
        }
    }
}
```

```

    }
}

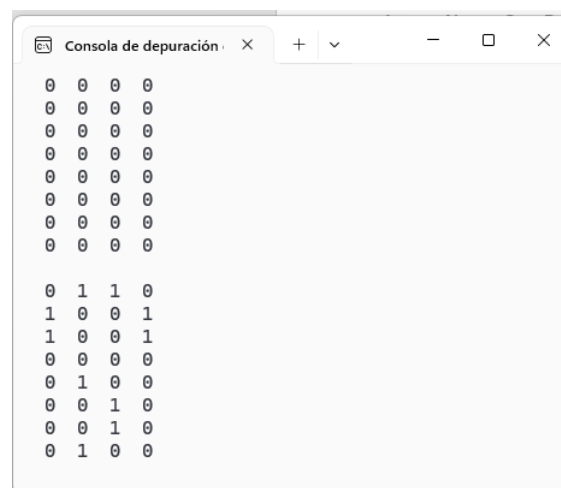
bool b;

prt(m);    cout << endl;
for (int i = 0; i < 10;i++)
{
    bool b = 0;
    while (b == 0)
    {
        b = booking(m, (rand() * 8) / RAND_MAX, (rand() * 4) / RAND_MAX);
    }
}
prt(m);
}

```

Develop a function called **"prt()"** that takes the boolean matrix created in the previous main function as its input argument.

Create a function called **"booking"** that takes the boolean matrix, along with the row and column numbers of the seat to be reserved, as input arguments. The function must check if the seat is available, and if so, modify its value to indicate that the seat is occupied. The function should return a boolean indicating whether the operation was successful.



1.12 Exercise 12: vector and matrix with classes

In this exercise, you will need the code from the previous exercise. Create the following class:

```

class passenger
{
    string name;
    int num;
};

void main()
{
    // rest of the code
    prt(m);
    prt(m,p);
}

```

Lesson Nº 9 - C++ Programming Exercises

In a similar way to how you worked with the boolean matrix, you need to create a matrix of **passenger**. Then, you should request the data every time a reservation is made.

You must also create a function **"prt()"** to display the results of the **passenger** matrix as shown below. If the seat is empty, simply display "...".



```
Consola de depuración de l x + - □ x
Name: ytr
Code: 9
Name: ghj
Code: 7
0 1 1 0
1 0 0 1
1 0 0 1
0 0 0 0
0 1 0 0
0 0 1 0
0 0 1 0
0 1 0 0
... uut, 10 aaa, 1 ...
ytr, 9 ... hgg, 6
dsdas, 5 ... ddd, 4
...
... ccc, 3 ...
... rer, 8 ...
... fff, 5 ...
... ghj, 7 ...
```